Sonlanma Problemi Üzerine Bir İnceleme
A Review on Halting Problem

Çağrı ÇAYCI
Computer Engineering Student, Gebze Technical University
May 2024

***Özetçe* —Bu belge, bilgisayar bilimlerinde temel bir kavram olan "Sonlanma Problemi"nin ne olduğu, neden çözülemeyen problemlerden biri olduğunu incelemeyi amaçlamaktadır. "Sonlanma Problemi", girdileri ve tanımı verilen bir bilgisayar programının, programının sonunda durup durmayacağını hiçbir şekilde belirleyemeceğimizi sorgular.**

Anahtar Kelimeler—Hesaplanabilirlik Teorisi, Algoritma, Turing Makinesi, İndirgeme Yöntemiyle İspat, Köşegenleştirme Yöntemiyle İspat.

***Abstract*—This document aims to examine what the "Halting Problem", a fundamental concept in computer science, is and why it is one of the unsolvable problems. The "Halting Problem" questions whether we can in any way determine whether a computer program, given its inputs and definition, will stop at the end of its program.**

Keywords—Computability Theory, Algorithm, Turing Machine, Proof by Reduction, Proof by Diagonalization.

INTRODUCTION [1], [2]

People might believe that as computers become more powerful, they will be able to solve every problem. However, this is not entirely true. Despite the increasing capabilities of computers, they are fundamentally limited, as demonstrated by the Halting Problem.

The Halting Problem is a fundamental concept in computability theory, proved by Alan Turing in 1936. Turing introduced the concept of a theoretical computing device known as the Turing Machine which can compute anything that is computable. Additionally, he claims that no general algorithm exists which can determine whether any given program will halt or continue to run forever for a particular input even memory and time requirements are neglected. This problem is undecidable meaning it is not possible to create universal algorithm that solves the Halting Problem for all possible program-input pairs. To illustrate the concept, consider two example programs:

Algorithm 1 An example for program never halts.

```
while true do
end while
```

Algorithm 2 An example for program halts

```
print "Hello World"
```

Deciding above programs halt or run forever is simple, the first program (Algorithm 1) enters an infinite loop and never halts. The second program (Algorithm 2) prints a message and then terminates. However, determining whether more complex programs halt or run forever is a significantly more challenging problem. For instance, consider the following program, referred to as the Collatz Conjecture (Algorithm 3). It posits that any number, when subjected to this algorithm, will eventually be reduced to 1. While numerous calculations support the conjecture's validity, it remains unproven that no number can persistently remain above 1. Consequently, it is undetermined whether the program enters an infinite loop or terminates.

Algorithm 3 Collatz Conjecture [8]

```
function collatz(n)
while n > 1 do
    show n
    if n is odd then
        n ← 3n + 1
    else
        n ← n/2
    end if
end while
show n
```

The significance of the Halting Problem stems from its profound implications for computation. This concept asserts that there are problems which no algorithm can solve. It underscores the critical realization that certain applications are inherently beyond the realm of flawless performance in programming.

In summary the Halting Problem highlights the limitations in even the most powerful computing. While simple examples can be easily distinguished as stalling or non-halting, the complexity of real-world programs makes the problem undecidable by any universal execution. The proof of Halting problem will be examined in this paper.

BODY

To understand the Halting Problem, it's essential to understand the concept of determining whether a given program will halt (terminate) or run indefinitely for a particular input. Formally, the problem asks whether there exists a universal algorithm that can decide, for any arbitrary program and input, whether the program will eventually halt when executed with that input.

The examination of the Halting Problem reveals the boundaries of algorithmic predictability and elucidates the essence of computation.

A. Informal Definition

To explain the goal informally, the following programs must be examined.

Algorithm 4 Program

```
Input: X
Y ← 0
while Y ≠ X do
    Y ← Y + 1
end while
```

Algorithm 5 Halt

```
Input: Program, X
if Program halts for X then
    return true
else
    return false
end if
```

In the first program, named "Program", the loop will either run forever or halt depending on the input. For example, if the input "X" is 10, the loop iterates 10 times and the program halts. However, if "X" is -1, the program will run forever because the loop condition will never be met.

The second program, named "Halt", is designed to determine whether the first program halts for a given input. The essence of the halting problem is to examine the possibility of creating such an algorithm, "Halt", that can universally decide whether any arbitrary program will halt or run indefinitely for any given input. The challenge is to prove whether such a universal algorithm can exist.

B. Formal Definition

The formal definition of the Halting Problem include a set as following. It consist of pairs of $\langle M, w\rangle$ where M represents a Turing Machine and w represents the given input to that Turing Machine.

$$\{\langle M, w\rangle \mid M \text{ is a Turing machine and } M \text{ halts on input } w\}$$

The Halting Problem queries whether there is a general algorithm that can predict, for any given Turing Machine and input pair, whether the Turing Machine will halt when run with that input.

C. Theorem

To prove the unsolvability of the Halting Problem, we need to carefully analyze the proposition. Two common methods for proving this are Proof by Reduction and Proof by Diagonalization.

$$\text{HALT.} = \{\langle M, w\rangle \mid M \text{ is a TM and } M \text{ halts on input } w\}$$

The theorem is proposed that HALT is undecidable. Thus, there is no TM that halts on every input.

D. Proof by Reduction [3], [4], [6]

Proof by reduction involves reducing a known undecidable problem to the Halting Problem, thereby proving that the Halting Problem is also undecidable.

Acceptance Problem for Turing Machine is expressed as follows and it is proved as undecidable;

$$A_{\text{TM}} = \{\langle M, w\rangle \mid M \text{ is a TM and } M \text{ accepts input } w\}$$

To prove HALT is also undecidable, A_{TM} will be reduced to HALT. Based on Turing Machine M, consider a new machine f(M) as follows:

Algorithm 6 Run Turing Machine M on input x

```
1: Run M on x
2: if M accepts then
3:     Halt and accept
4: else
5:     Go into an infinite loop
6: end if
```

Observe that f(M) halts on input w if and only if M accepts w.

Suppose HALT is decidable. Then there is a Turing Machine H that always halts and $L(H) = HALT$. Consider the following program T;

Algorithm 7 T

```
1: Input: ⟨M, w⟩
2: Construct program f(M)
3: Run H on ⟨f(M), w⟩
4: if H accepts then
5:     Accept
6: else
7:     Reject
8: end if
```

T decides A_{TM}. But, it is proven that A_{TM} is undecidable, which gives the contradiction.

E. Proof by Diagonalization [3], [5]

TABLO I: Behaviour of all TMs for all inputs

	$\langle M_1\rangle$	$\langle M_2\rangle$	...	$\langle M_n\rangle$
M_1	$accept$	$reject$	...	$doesn't\ halt$
M_2	$\langle M_2\langle M_1\rangle\rangle$	$does'nt\ halt$	...	$\langle M_2\langle M_n\rangle\rangle$
⋮	⋮	⋮	⋱	⋮
M_x	$\langle M_x\langle M_1\rangle\rangle$	$\langle M_x\langle M_2\rangle\rangle$	...	$\langle M_x\langle M_n\rangle\rangle$
⋮	⋮	⋮	⋱	⋮
M_n	$\langle M_n\langle M_1\rangle\rangle$	$\langle M_n\langle M_2\rangle\rangle$	...	$reject$

Let's suppose that A_{TM} is decidable. This implies the existence of a Turing Machine, denoted as H, which is capable of deciding A_{TM}. The goal of H is to determine whether it should accept or reject when given a Turing Machine M_i and an input M_j. To achieve this, H utilizes a decision table, outlined in Table I. If the table I indicates acceptance for a given pair (M_i, M_j), then H returns accept. Conversely, if the

table I indicates rejection or the machine does not halt, H returns reject. Therefore, Table II is identical to Table I excepts where Table I has "doesn't halt" but Table I has "reject".

TABLO II: Behaviour of H on input $\langle M_{\mathrm{i}} \langle M_{\mathrm{j}} \rangle \rangle$

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	...	$\langle M_{\mathrm{n}} \rangle$
M_1	$accept$	$reject$	...	$reject$
M_2	$\langle M_2 \langle M_1 \rangle \rangle$	$reject$	...	$\langle M_2 \langle M_{\mathrm{n}} \rangle \rangle$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
M_{n}	$\langle M_{\mathrm{n}} \langle M_1 \rangle \rangle$	$\langle M_{\mathrm{n}} \langle M_2 \rangle \rangle$	...	$reject$

Assuming the existence of a Turing Machine (TM) denoted as H, we can infer the presence of another TM capable of processing inputs in the format $\langle M \rangle$, where M represents the description of a TM. Let's label this second TM as D. Upon receiving an input $\langle M \rangle$, D executes a specific operation: it reverses the entry along the diagonal of Table II.

TABLO III: Behaviour of D on input $\langle M_{\mathrm{i}} \rangle$

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	...	$\langle M_{\mathrm{n}} \rangle$
M_1	$reject$	$accept$	...	...	$accept$

As Table I encompasses all TMs, one of them must correspond to TM M_{x}, denoted as D. Along the diagonal of Table I, $\langle M_{\mathrm{x}} \langle M_{\mathrm{x}} \rangle \rangle$ can be in one of three states: accept, reject, or doesn't halt. If $\langle M_{\mathrm{x}} \langle M_{\mathrm{x}} \rangle \rangle$ accepts in Table I, it would consequently accept in Table II. However, due to reverse logic, it should then reject in Table III. Additionally, if $\langle M_{\mathrm{x}} \langle M_{\mathrm{x}} \rangle \rangle$ is either reject or doesn't halt, it would reject in Table II. Finally, it would accept in Table III. Consequently, M_{x} cannot be equal to D, leading to a contradiction.

CONCLUSION

In conclusion, Halting Problem, formulated by Alan Turing in 1936, is one of the fundamental concepts of computability theory. It states that there is no general algorithm to determine whether a given program will halt or run indefinitely for all possible inputs. It states that there are inherit limitations for a computer can compute, no matter how clever an algorithm or a program is. Additionally, it indicates that some problems are not solvable by their nature. There is a limit of computation.

This paper explores the concept of the Halting Problem, delving into its definition, proposed solutions, various techniques for proving its undecidability, and the application of these techniques in demonstrating its unsolvability.

In software development, the Halting Problem underscores the challenge of ensuring programs terminate correctly and efficiently. Techniques such as static analysis, testing, and code review help mitigate non-termination risks, though complete assurance remains unattainable. Moreover, tools for program verification, model checking, and software testing often encounter the Halting Problem, prompting the development of sophisticated algorithms and heuristics to analyze program behavior, correctness, and security. Thus, the Halting Problem continues to shape theoretical and practical approaches in computing, reminding us of the limits and guiding innovation in the field. [7]

REFERENCES

[1] Wikipedia contributors, "Halting problem", Wikipedia, The Free Encyclopedia

[2] S. Lucas, "The origins of the halting problem," Journal of Logical and Algebraic Methods in Programming, vol. 121, pp. 100687, 2021. doi: https://doi.org/10.1016/j.jlamp.2021.100687.

[3] M. Sipser "Introduction to the Theory of Computation", 3rd ed., Cengage Learning, 2013, pp. 173-182.

[4] Agha-Viswanathan. (2010). CS373 Undecidability Reductions The Halting Problem Informal Overview Definition and Properties.

[5] D. Gusfield, Theory of Computation, Lecture 15: Proof by Diagonalization that ATM (Halting Problem) is Not Decidable, 2012, UC Davis.

[6] H. Porter, Theory of Computation, Portland State University, 2014. Lecture on Halting Problem: A Proof by Reduction.

[7] D. Andre, "What is the Halting Problem?", January, 2024.

[8] Esolangs contributors, "Collatz Sequence", Esolangs Wiki